

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

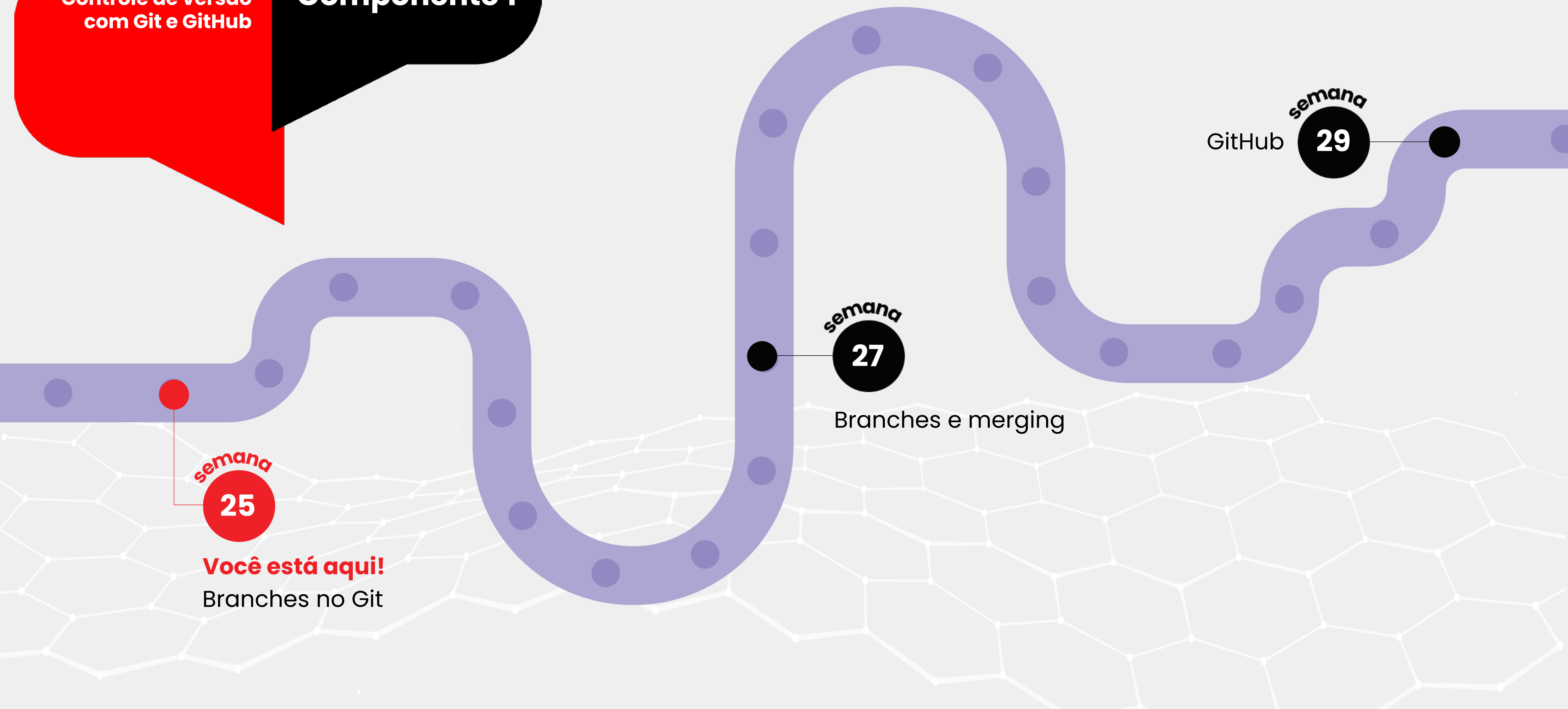
Branches no Git

Introdução ao Git e conceitos básicos de branches

Código da aula: [DADOS]ANO1C1B4S25A1

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1



Controle de versão
com Git e GitHub

Mapa da
Unidade 8
Componente 1

Você está aqui!

Branches no Git

**Introdução ao Git e conceitos
básicos de branches**

Código da aula: [DADOS]ANO1C1B4S25A1

25



Objetivos da aula

- Introduzir e utilizar ferramentas de desenvolvimento de software como Git e GitHub.



Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou internet.



Duração da aula

50 minutos.



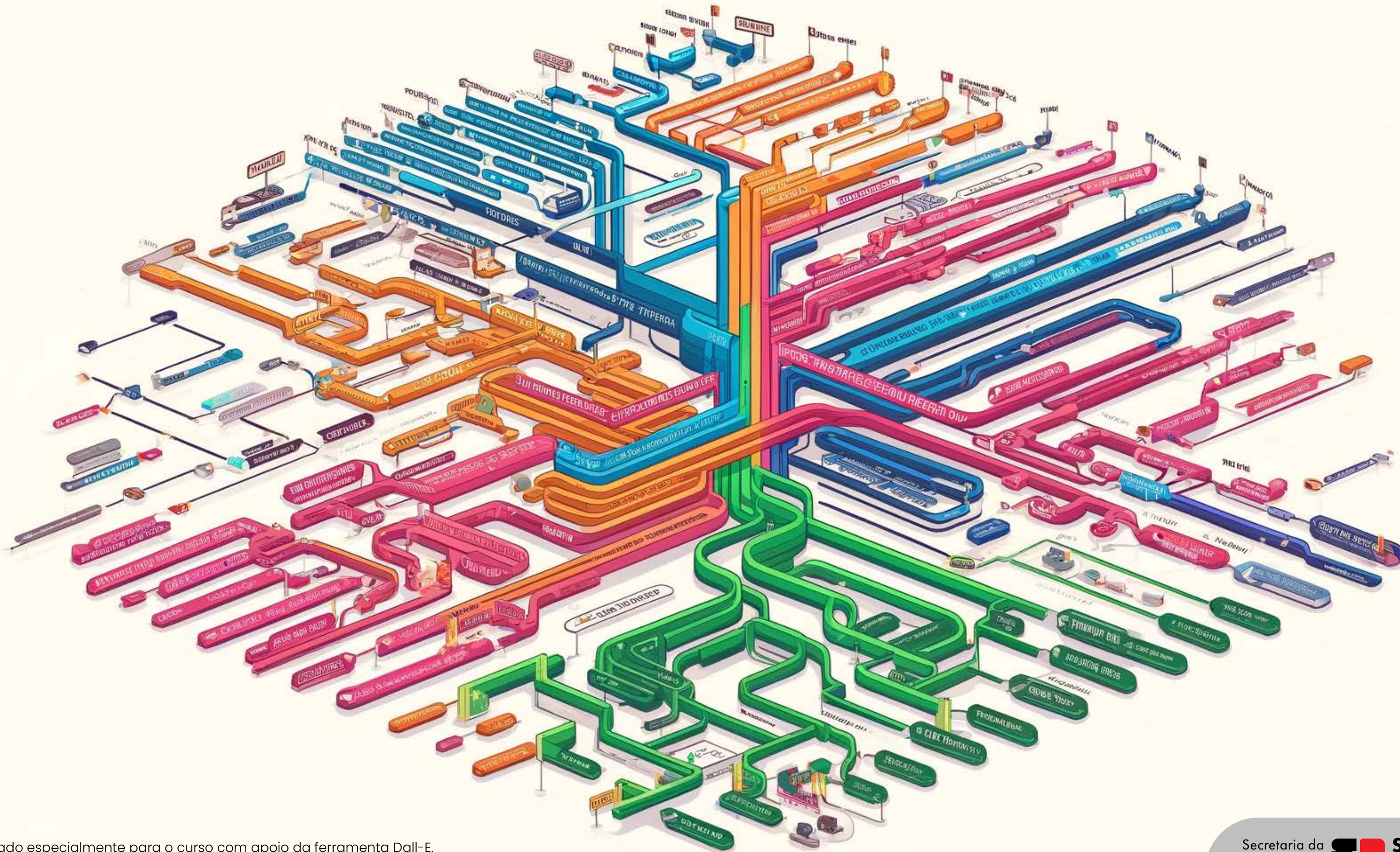
Competências técnicas

- Usar ferramentas de desenvolvimento de software como Git e GitHub;
- Compreender e dominar técnicas de manipulação de dados.



Competências socioemocionais

- Analisar informações de forma crítica e tomar decisões;
- Avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados;
- Trabalhar em equipes multifuncionais, colaborando com colegas, gestores e clientes.



Elaborado especialmente para o curso com apoio da ferramenta Dall-E.

Primeiras **ideias**

O que representa cada linha dessa imagem?

Por que será que há tantas linhas diferentes?

Como será que essas linhas podem se unir novamente?

Ponto de partida



© Getty Images

Imagine que você está trabalhando em uma equipe de desenvolvimento de software e todos os membros estão fazendo mudanças no mesmo código-fonte.

Como garantir que essas mudanças não entrem em conflito e que todos possam colaborar eficientemente?

Construindo o **conceito**

História e importância do Git

- ▶ É um sistema de controle de versão distribuído criado por Linus Torvalds em 2005;
- ▶ Foi desenvolvido, originalmente, para gerenciar o código-fonte do kernel do Linux;
- ▶ Tornou-se, rapidamente, um dos sistemas de controle de versão mais populares devido à sua eficiência, flexibilidade e robustez;
- ▶ Permite que múltiplos desenvolvedores trabalhem simultaneamente em diferentes partes de um projeto, sem interferir uns nos outros, facilitando a colaboração e o rastreamento de alterações ao longo do tempo.

Fonte: CHACON; STRAUB, 2014.

Construindo o conceito

Conceitos básicos de Git

O Git opera como um sistema distribuído de controle de versão, em que cada desenvolvedor possui uma cópia completa do repositório, incluindo todo o histórico de mudanças.

Alguns conceitos-chave incluem:

Repositório (repo)

Local onde o histórico do projeto é armazenado.

Commit

Registro de mudanças feitas no código.

Branch

Linha de desenvolvimento independente.

Merge

Combinação de mudanças de diferentes branches.

Clone

Cópia de um repositório.

Pull / pull request

Obtenção de mudanças de um repositório remoto.

Elaborado especialmente para o curso com imagem © Getty Images.

Construindo
o **conceito**

O que são branches no Git?

Os **branches** são versões paralelas de um repositório Git, permitindo que desenvolvedores trabalhem em diferentes funcionalidades ou correções de bugs de forma isolada.



Tome nota

O **branch principal** é geralmente chamado de **main** ou **master**, enquanto outros branches são criados para desenvolvimento de novas features, correções de bugs etc.

Construindo
o **conceito**

Criação e gerenciamento de branches

	Comandos
Criar um branch	git branch nome-do-branch
Mudar para um branch	git checkout nome-do-branch ou git switch nome-do-branch
Listar branches	git branch
Deletar um branch	git branch -d nome-do-branch

Elaborado especialmente para o curso.

Construindo
o **conceito**

Merging e resolução de conflitos

Merging é o processo de combinar mudanças de diferentes branches. Isso pode ser feito com o comando:

- **Merge simples**

git merge nome-do-Branch



Atenção

A **resolução de conflitos** ocorre quando mudanças conflitantes são feitas no mesmo arquivo em diferentes branches. O Git tenta automaticamente resolver, mas se não conseguir, requer intervenção manual.

Construindo
o **conceito**

Exemplo

Passo a passo para trabalhar com arquivos `.py` e Git no **Jupyter Notebook**:

1

Criação de arquivos `.py` no Jupyter Notebook

Você pode criar arquivos Python dentro do Jupyter Notebook usando o comando `%%writefile` em uma célula de código.

2

Executando comandos Git no Jupyter Notebook

Você pode executar comandos de Git no Jupyter Notebook prefixando os comandos com um ponto de exclamação (!).

Construindo o **conceito**

Exemplo

1. Criação do repositório Git e arquivo .py

a. Crie um novo repositório Git:

```
!git init my_git_project
```

b. Crie um arquivo Python dentro do Jupyter Notebook:

```
%%writefile my_git_project/sum.py  
def add_numbers(a, b):  
    return a + b  
  
print(add_numbers(2, 3))
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplo

1. Criação do repositório Git e arquivo .py
- c. Adicione o arquivo ao repositório Git e faça o primeiro commit:

```
!cd my_git_project && git add sum.py  
!cd my_git_project && git commit -m "Initial commit"
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplo

2. Fazendo alterações e commits

a. Atualize o arquivo Python com uma nova função:

```
%%writefile my_git_project/sum.py
def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b

print(add_numbers(2, 3))
print(subtract_numbers(5, 2))
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo
o **conceito**

Exemplo

2. Fazendo alterações e commits

b. Adicione as mudanças ao repositório Git e faça um novo commit:

```
!cd my_git_project && git add sum.py  
!cd my_git_project && git commit -m "Added subtract_numbers function"
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplo

3. Criando e alternando entre branches

a. Crie um novo branch:

```
!cd my_git_project && git branch nova-funcionalidade
```

b. Mude para o novo branch:

```
!cd my_git_project && git checkout nova-funcionalidade
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplo

3. Criando e alternando entre branches

c. Adicione uma nova função no arquivo Python:

```
%%writefile my_git_project/sum.py
def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b

def multiply_numbers(a, b):
    return a * b

print(add_numbers(2, 3))
print(subtract_numbers(5, 2))
print(multiply_numbers(3, 4))
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplo

3. Criando e alternando entre branches

d. Adicione e faça commit das mudanças no novo branch:

```
!cd my_git_project && git add sum.py  
!cd my_git_project && git commit -m "Added multiply_numbers function"
```

e. Volte para o branch principal e faça o merge:

```
!cd my_git_project && git checkout main  
!cd my_git_project && git merge nova-funcionalidade
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Exemplo

4. Resolução de conflitos

a. Simule um conflito criando outro branch e fazendo mudanças conflitantes.

Branch 1:

```
!cd my_git_project && git checkout -b branch1
```

```
%%writefile my_git_project/sum.py
def add_numbers(a, b):
    return a + b

def divide_numbers(a, b):
    return a / b # Adicionado no branch1

print(add_numbers(2, 3))
print(divide_numbers(6, 2))
```

```
!cd my_git_project && git add sum.py
```

```
!cd my_git_project && git commit -m "Added divide_numbers function in branch1"
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o conceito

Exemplo

4. Resolução de conflitos

b. Simule um conflito criando outro branch e fazendo mudanças conflitantes.

Branch 2:

```
!cd my_git_project && git checkout main  
!cd my_git_project && git checkout -b branch2
```

```
%%writefile my_git_project/sum.py  
def add_numbers(a, b):  
    return a + b  
  
def divide_numbers(a, b):  
    if b != 0:  
        return a / b  
    else:  
        return "Division by zero error" # Adicionado no branch2  
  
print(add_numbers(2, 3))  
print(divide_numbers(6, 0))
```

```
!cd my_git_project && git add sum.py  
!cd my_git_project && git commit -m "Added divide_numbers function with zero check in branch2"
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo
o **conceito**

Exemplo

4. Resolução de conflitos

c. Tente fazer o merge e resolva o conflito manualmente:

```
!cd my_git_project && git checkout main  
!cd my_git_project && git merge branch1  
!cd my_git_project && git merge branch2  
# Conflito detectado em sum.py
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo o **conceito**

Exemplo

4. Resolução de conflitos

d. Edite manualmente o arquivo **sum.py** para resolver o conflito:

```
%%writefile my_git_project/sum.py
def add_numbers(a, b):
    return a + b

def divide_numbers(a, b):
    if b != 0:
        return a / b
    else:
        return "Division by zero error"

print(add_numbers(2, 3))
print(divide_numbers(6, 0))
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.

Construindo
o **conceito**

Exemplo

4. Resolução de conflitos

e. Finalize o merge após resolver o conflito:

```
!cd my_git_project && git add sum.py  
!cd my_git_project && git commit -m "Resolved conflict by adding zero division check"
```

Elaborado especialmente para o curso com a ferramenta Jupyter Notebook.



Vamos
fazer um
quiz

O que é um branch no Git?

Linha de desenvolvimento
principal.

Linha de desenvolvimento
independente.

Histórico de commits.

Repositório clonado.



Vamos
fazer um
quiz

Qual comando cria um novo branch?

git checkout nome-do-branch

git branch nome-do-branch

git merge nome-do-branch

git commit nome-do-br



Vamos
fazer um
quiz

O que é um merge no Git?

Combinação de mudanças de diferentes branches.

Criação de um novo branch.

Exclusão de um branch.

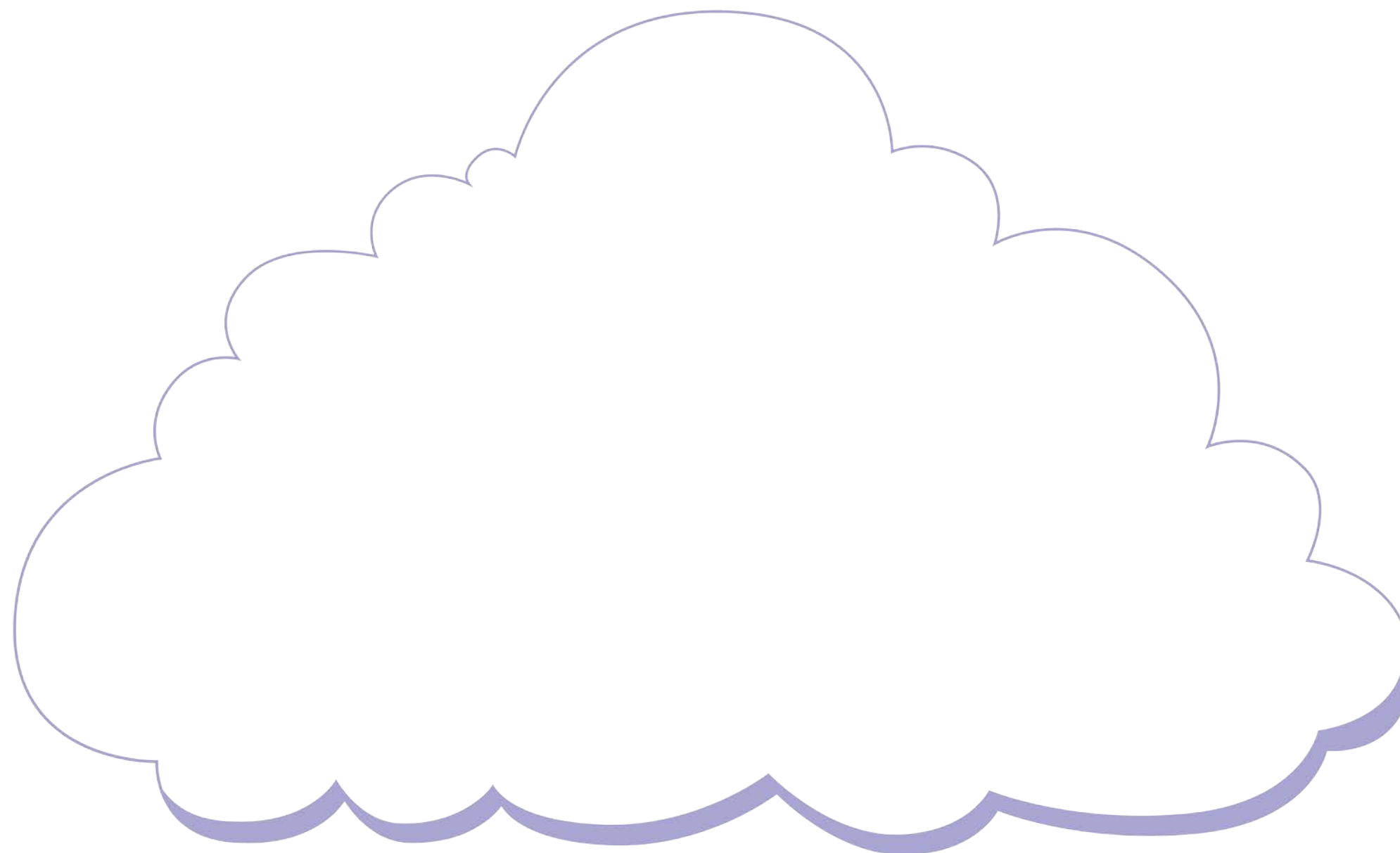
Clone de um repositório.

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**





© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** O Git é um sistema de controle de versão distribuído criado por Linus Torvalds em 2005, permitindo colaboração eficiente entre desenvolvedores, ao armazenar o histórico completo de mudanças em um repositório.
- 2** Os branches no Git são linhas de desenvolvimento independentes que permitem a criação de novas funcionalidades ou correções de bugs sem interferir no código principal, facilitando a colaboração.
- 3** O processo de merge combina mudanças de diferentes branches, e conflitos devem ser resolvidos manualmente, garantindo a integridade do código, ao integrar diferentes linhas de desenvolvimento.

Saiba mais

Você usa a versão atualizada do Git Checkout?

As novas versões do Git simplificaram o gerenciamento de branches com os comandos git restore e git switch. Descubra como essa mudança facilita o seu dia a dia e domine o Git de forma eficiente!

DIAS, V. Entenda os comandos git restore e switch. Alura, [s.d.].

Disponível em: <https://cursos.alura.com.br/extra/alura-mais/entenda-os-comandos-git-restore-e-switch-c99>.

Acesso em: 12 jul. 2024.

Referências da aula

BELL, P.; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2014.

CHACON, S.; STRAUB, B. *A short history of Git*. In: CHACON, S.; STRAUB, B. *Pro Git*. [s.l.]: Apress, 2014. Disponível em: <https://git-scm.com/book/en/v2>. Acesso em: 24 jul. 2024.

GITHUB. *Documentação de introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started>. Acesso em: 12 jul. 2024.

Identidade visual: imagens © Getty Images.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**